

PHASE 4 OF 8

AI Integration with Gemini

Milestone 3: how the Gemini client is configured, what the prompt must enforce, and what the output looks like.

- Milestone 3
- Gemini Client
- Prompting
- Output

Milestone 3 — AI Integration

Activity 3.1: Gemini Client

backend/ai_core/gemini_generator.py — Gemini client call:

```
from google import genai
from google.genai import types

class GeminiDocumentGenerator:
    def __init__(self, api_key, model):
        self.client = genai.Client(api_key=api_key)
        self.model = model

    def generate_document(self, prompt):
        response = self.client.models.generate_content(
            model=self.model,
            contents=prompt,
            config=types.GenerateContentConfig(
                temperature=0.2,
                max_output_tokens=8192
            )
        )
        return response.text
```

The generator creates a structured prompt from the user’s requirements and sends it to the configured Gemini model. The API credential is supplied through configuration and is not hard-coded into the source.

Generation settings

Setting	Value	Effect
model	gemini-3.5-flash-lite	Set through GEMINI_MODEL; a fast, lightweight Gemini model.
temperature	0.2	Low randomness, so wording stays formal and consistent between runs.
max_output_tokens	8192	Leaves room for long, multi-clause documents without truncation.

About the excerpts

The code excerpts in this documentation are simplified. The route passes the individual form fields to the generator, which assembles them into the prompt before the model call shown above.

Activity 3.2: Prompt Requirements

The prompt is what turns a general-purpose model into a careful drafting assistant. It must instruct the model to:

- Use **only information supplied by the user**.
- Produce a **professional legal-document draft**.
- Use **numbered sections and signature blocks** where appropriate.
- **Avoid inventing user-specific facts**.
- Include an **informational legal disclaimer**.

Why each requirement matters

Requirement	Risk it prevents	Visible effect
Use only supplied information	Invented names, amounts or dates	Placeholders appear where data was not provided
Professional draft	Casual or ambiguous wording	Formal clauses such as “IN WITNESS WHEREOF”
Numbered sections + signatures	Hard-to-review, unstructured text	Sections 1, 2, 3... and signature blocks
No invented facts	Misleading legal statements	Addresses left as [ADDRESS TO BE COMPLETED]
Legal disclaimer	Drafts mistaken for legal advice	Notice reminding users to seek professional review

Prompt inputs

The generator combines the six form values into one structured instruction:

Input	Demonstration value
Document type	Freelance Work Contract
Parties	Arun Kumar (Freelancer), BrightTech Solutions (Client)
Effective date	2026-09-25
Jurisdiction	Tamil Nadu, India
Terms	Website development; ₹50,000; 50% upfront / 50% on completion; 30 days; confidentiality; 15 days’ notice
Additional instructions	Extra drafting or formatting requirements (optional)

Generated output in the interface

After the user clicks **Generate Document**, the interface confirms success and shows the draft in two places: a formatted preview on the left and an editable copy on the right.

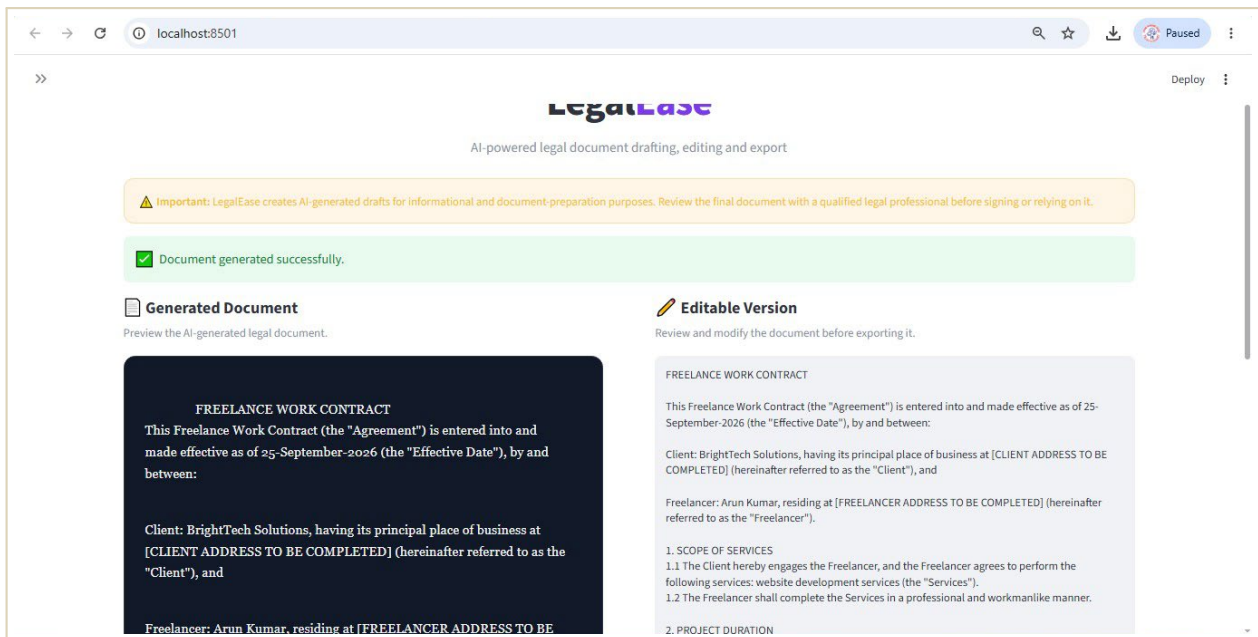


Figure 3 — AI-generated legal document and the “Document generated successfully” message.

What the demonstration output shows

- The draft opens with the title **FREELANCE WORK CONTRACT** and states the effective date supplied by the user.
- Both parties are named exactly as entered, in the roles the user gave them (Client and Freelancer).
- Details the user did not provide — such as street addresses — are left as clear placeholders in square brackets rather than guessed.
- The text is organised into numbered clauses (scope of services, duration, payment terms, confidentiality, governing law, miscellaneous) and ends with signature blocks.

Reliability

If Gemini cannot be reached, the route returns a 502 error and the frontend’s `raise_for_status()` call surfaces the failure. The frontend waits up to 120 seconds for a response, which is generous enough for long documents.